

CCDV-F Flashcards

Format: Q → answer. Cover with your hand, work through each section.

Agents and Workflows

- **Q:** What's the core distinction between a workflow and an agent? → Workflow: LLMs/tools follow a predefined code path (orchestration in your code). Agent: the LLM dynamically directs its own process and tool use.
- **Q:** Decision rule for workflow vs. agent when you're unsure? → Start with an agent, extract deterministic workflow patterns as they emerge from real usage — don't guess a workflow structure up front.
- **Q:** Name the five workflow patterns from Anthropic's "Building Effective AI Agents." → Prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.
- **Q:** Prompt chaining vs. routing? → Chaining: sequential LLM calls, each processing the prior output. Routing: classify input, send to a specialized handler per category.
- **Q:** Parallelization's two sub-modes? → Sectioning (independent subtasks, for speed) and voting (multiple attempts, for confidence).
- **Q:** Which pattern is the bridge between pure workflow and pure agent? → Orchestrator-workers — decomposition is dynamic (agent-like), each worker's task is scoped (workflow-like).
- **Q:** Supervisor pattern in one sentence? → A central supervisor delegates to worker agents and reviews/aggregates their output, typically without executing the work itself.
- **Q:** Three benefits subagents provide regardless of architecture? → Context isolation, parallelization, specialization (narrow system prompt + scoped tools).
- **Q:** What does a subagent NOT inherit from its parent? → The parent's conversation history, tool results, or system prompt — it starts with a fresh context window.
- **Q:** What DOES a subagent receive? → Its own `AgentDefinition.prompt`, the Agent tool's prompt string, and project CLAUDE.md (via `settingSources`).
- **Q:** `max_turns` vs. `max_budget_usd` in the Agent SDK? → `max_turns` caps tool-use round trips; `max_budget_usd` caps spend (covers subagent spend too).
- **Q:** What happens when a hit limit is reached? → `ResultMessage` returns with `error_max_turns` or `error_max_budget_usd` subtype instead of raising mid-loop.
- **Q:** Five `effort` levels and what they trade off? → low/medium/high/xhigh/max — trade reasoning depth for latency and token cost.
- **Q:** Self-hosted (Agent SDK) vs. Anthropic-hosted (Managed Agents) — who handles what? → Self-hosted: you provision servers, OAuth, secrets, retries, endpoints. Managed: Anthropic handles all of it.
- **Q:** What's the middle-ground deployment option? → Self-hosted sandboxes — orchestration stays on Anthropic's side, tool execution moves into your infrastructure.
- **Q:** `PreToolUse` hook's most important capability? → Can deny a tool call outright before it executes — a deterministic, code-level guarantee a prompt-level rule can't provide.
- **Q:** Hooks run where, and does that cost context? → In your application process, not inside the model's context window — no token cost.
- **Q:** Programmatic vs. filesystem-based subagent definition — which wins if both define the same name? → Programmatic (`agents` param in `query()`) takes precedence over a filesystem `.claude/agents/` file of the same name.
- **Q:** What does the `Workflow` tool solve that turn-by-turn subagent delegation doesn't? → Orchestrating dozens-to-hundreds of agents via a script executed outside the conversation's own context.

- **Q:** LangGraph, PydanticAI, Strands — one differentiator each? → LangGraph: explicit graph-based state control. PydanticAI: Python type safety/validation. Strands: AWS/Bedrock-native, model-driven.
- **Q:** Anthropic's own reported cost multiplier for an orchestrator-worker run vs. a single-agent chat? → Roughly 15x the tokens — a lead plus workers each carry their own context, input, and output.
- **Q:** When does the orchestrator-worker multiplier NOT pay off? → On tightly-coupled work where each step depends on the last (coding is the standard example) — subagents mostly wait on each other, so you pay the fan-out cost without the parallel benefit.
- **Q:** Two cost mitigations when you do use orchestrator-workers? → Use a more capable model only for the lead, cheaper models for workers; watch for a runaway subagent or oversized tool result pushing well past the expected multiplier.
- **Q:** Raw Messages API loop vs. Agent SDK vs. Managed Agents — who runs the loop in each? → Raw loop: your code, every iteration. Agent SDK: the SDK, inside your process, you still execute tools. Managed Agents: Anthropic runs the loop and sandbox; you send events and stream results.
- **Q:** What single constraint rules out Managed Agents regardless of operational fit? → A PHI or Zero Data Retention requirement — Managed Agent sessions are stateful and stored server-side, not currently ZDR/HIPAA-BAA eligible.
- **Q:** The question that determines whether a human-in-the-loop gate belongs at a given step? → What is the worst outcome if this step runs without a human check?
- **Q:** Three agent memory scopes, and when each fits? → In-context (one continuous session, fits comfortably), external storage (same task across many separate shorter sessions), stateless (fully independent jobs, no persistence needed).
- **Q:** Why did an agent that worked in dev (one long session) fail in production (many short sessions)? → Injected history accumulated across sessions and exceeded 40K tokens before the first tool call — development and production used different session *shapes*, and in-context memory handles them very differently.

Applications and Integration

- **Q:** Functional vs. infrastructure requirements? → Functional: what the app does (task, inputs, output shape). Infrastructure: what the system must satisfy regardless of feature (latency budget, throughput, residency, cost ceiling).
- **Q:** Why does the "maintain" SDLC phase never really finish for a Claude app? → Model behavior can shift on version bump and prompts drift as usage evolves — eval suites are what makes ongoing maintenance tractable.
- **Q:** Is the Messages API stateful or stateless per request? → Stateless — your application owns and resends the full conversation state each call (unless using a session-store layer on top).
- **Q:** Three image source types in the Vision API, and the one to use for multi-image conversations? → `base64`, `url`, `file` (Files API) — use `file` / `file_id` to avoid re-sending full image bytes every turn.
- **Q:** Extended thinking vs. adaptive thinking — which models get which? → Fable 5/Opus 5/Sonnet 5: adaptive thinking. Haiku 4.5: extended thinking. Not the same feature, not interchangeable.
- **Q:** Two concrete API differences when invoking Claude via Google Cloud vs. the first-party API? → Model specified in the endpoint URL (not request body), and `anthropic_version` is a required body field.
- **Q:** Amazon Bedrock's two endpoint types, and why they matter? → Global (dynamic routing, max availability) vs. regional (guaranteed routing through a specific geography) — a data-residency/compliance lever the first-party API doesn't expose the same way.
- **Q:** Sample Question 1 style scenario — 10,000 documents, overnight, cost-sensitive → which API? → Message Batches API (up to 24h, lower per-token cost) — not synchronous-in-parallel, not just shrinking `max_tokens`.

- **Q:** What's the risk of code-reviewing only application logic and not tool-schema/prompt changes? → A tool schema change is a breaking-change risk for every caller of that tool, but it won't show up in a review scoped to "just the code."
- **Q:** How does Claude get persistent instructions differently across Claude Code vs. the raw API? → Claude Code: CLAUDE.md hierarchy, re-injected automatically. Raw API: only the `system` parameter you explicitly set per request — no implicit persistence.
- **Q:** What is a Claude Code plugin, structurally? → A distributable bundle of commands/agents/skills/hooks described by a `.claude-plugin/plugin.json` manifest.
- **Q:** What can a plugin marketplace auto-install? → A plugin's declared dependencies (from `plugin.json`, possibly overridden by marketplace-entry fields).
- **Q:** Why pin a model version in production? → Protects against an unannounced behavior shift on model upgrade; every Claude model ID is a pinned snapshot, dated or (4.6+) dateless-but-pinned.
- **Q:** Should prompts be version-controlled like code? → Yes — a "small wording tweak" can measurably shift output distribution; treat prompt/config changes with the same rigor (VCS, review, eval validation) as code changes.
- **Q:** A business problem like "help agents answer faster" — is that a requirement? → No — functional/infrastructure requirements are *derived* from it, stated specifically enough to check (e.g. "classify into one of four queues; never auto-send without approval").
- **Q:** Four infrastructure requirements that most often decide the deployment platform? → Latency, scale, residency, identity — mostly not stated in the business problem, derived by asking what it implies.
- **Q:** The seven systems-lifecycle phases for a Claude application? → Requirements → Design → Build → Test → Deploy → Operate → Iterate.
- **Q:** What is a "gate" between lifecycle phases, and give an example. → The decision to move to the next phase only once a condition holds — e.g. don't move design→build until the platform satisfies residency; don't move deploy→production until the new version clears the eval against its baseline.
- **Q:** Six places a Claude workload can run? → First-party API, Claude Platform on AWS, Claude in Amazon Bedrock, Claude on Bedrock (legacy), Google Vertex AI, third-party (e.g. Microsoft Foundry).
- **Q:** Microsoft Foundry's residency gotcha? → Two hosting forms with different residency — "Hosted on Azure" runs inference end-to-end on Azure; "Hosted on Anthropic" does not satisfy EU regional residency. Must confirm per model, not per platform.
- **Q:** Three dimensions for comparing deployment platforms, and how to measure each correctly? → Latency (from the customer's actual region, not your laptop), compliance (against their existing certification, usually pass/fail), cost (total per call including egress/integration, not just token price).
- **Q:** What does "packaging for reuse" actually mean for a working build? → Separating customer-specific values into documented, parameterized configuration with defaults, and bundling the eval suite — so a future team configures the asset instead of rebuilding it.

Claude Code

- **Q:** Claude Code's five core components? → Rules (CLAUDE.md), Skills, Commands, Agents, Agent Memory.
- **Q:** settings.json precedence, highest to lowest? → Managed > CLI > Local > Project > User.
- **Q:** What's the one exception to settings.json override precedence? → `permissions` (allow/deny/ask) accumulate across all applicable scopes instead of the highest scope winning outright.
- **Q:** Six Claude Code permission modes, from most to least restrictive-by-default? → `default` (prompts nearly everything) → `plan` (read-only until approved) → `acceptEdits` (auto file edits + FS commands in working dir) → `auto` (classifier-reviewed) → `dontAsk` (allow-list only, else deny) → `bypassPermissions` (everything, isolated environments only).

- **Q:** Which permission mode uniquely skips the protected-path guard the others keep? → `bypassPermissions`.
- **Q:** CLAUDE.md vs. a rules instruction file — what decides scope? → CLAUDE.md loads every session unconditionally. A rules file loads only for matching files via a `paths` glob in its YAML frontmatter — the scoping comes from frontmatter, not directory placement.
- **Q:** What's the failure mode when CLAUDE.md keeps growing? → Dilution — a correct rule buried among hundreds of other lines gets a smaller share of the model's attention and can still be violated even though it's technically present.
- **Q:** Which built-in subagents skip CLAUDE.md and git status, and why does it matter? → `Explore` and `Plan` — optimized for speed. If a project rule silently doesn't apply to a delegated task, this is often why.
- **Q:** Skills across four runtimes — what's the Agent SDK gotcha specifically? → A skill that worked in Claude Code does nothing under the SDK because `settingSources` / `setting_sources` was never set explicitly — there's no safe default to rely on.
- **Q:** Why is Managed Agents' skill loading different from the other three runtimes? → Skills are attached when defining the agent as an API resource (server-side), not discovered from a filesystem — no local discovery step at all.
- **Q:** How are plugin commands namespaced, and why does it matter? → Automatically by plugin name (`/payments:run-tests`) — lets two plugins ship a same-named command without colliding, but renaming a plugin renames every command it ships.
- **Q:** What's the difference between a managed marketplace allowlist and `extraKnownMarketplaces`? → The allowlist restricts which marketplace sources users may add; `extraKnownMarketplaces` pushes a marketplace to all users automatically, without requiring the manual add command.
- **Q:** Why did a plugin install cleanly everywhere but only run on the author's machine? → An absolute path (`/Users/author/...`) in the skill's SKILL.md — install just copies files, execution resolves paths against whichever machine is running it. Fix: `$CLAUDE_PROJECT_DIR / ${CLAUDE_PLUGIN_ROOT}`.
- **Q:** What does `--bare` skip, and why use it in CI? → Skips auto-discovery of hooks, skills, plugins, MCP servers, auto memory, CLAUDE.md — for faster, deterministic CI runs. Tradeoff: no OAuth/keychain reads, must set `ANTHROPIC_API_KEY` explicitly.
- **Q:** `--output-format` options for headless mode? → `text` (default), `json` (structured, includes cost/session metadata), `stream-json` (newline-delimited streaming events).
- **Q:** What extra flag do you need with `stream-json` for token-level deltas? → `--include-partial-messages` (alongside `--verbose`).
- **Q:** Does a `.claude/commands/deploy.md` file behave differently from a `.claude/skills/deploy/SKILL.md`? → No — both create a `/deploy` invocation and work the same way; commands and skills are two authoring surfaces for the same mechanism.

Eval, Testing, and Debugging

- **Q:** 429 vs. 529 — same handling? → Yes, both retryable with exponential backoff (`rate_limit_error` vs. `overloaded_error`); 400/401/402/403 are not retryable as-is.
- **Q:** The core debugging question this domain tests? → Is the failure in the integration layer (malformed request, bad schema, hook denial) or in the model's output (wrong/incomplete response)?
- **Q:** How do you diagnose an integration-layer failure vs. re-prompting? → Inspect the raw request/response and the harness's event stream (SDK message types, `stream-json` events) — not by changing the prompt.
- **Q:** `stop_reason: "refusal"` or `"max_tokens"` on an otherwise-valid request — what should you look at? → What you asked for (the prompt/task), not your request-construction code.

- **Q:** Are Agent SDK `ResultMessage` error subtypes the same thing as HTTP error codes? → No — they describe why the *agent loop* stopped (turns/budget/execution/structured-output-retries exhausted), distinct from why a single API call failed.
- **Q:** Why write the eval *before* the feature? → Defining expected behavior up front forces you to define success before implementation — otherwise you tend to rationalize whatever the model happens to produce later.
- **Q:** Exact match vs. code-graded check vs. LLM-as-judge — pick the grader for each output shape. → One correct label/value → exact match. Structured/code output → code-graded check (format only). Open-ended quality → LLM-as-judge (needs calibration).
- **Q:** Why ask a judge for strengths/weaknesses/reasoning before the score? → Without reasoning-first, judge models drift toward a safe middle score (~6) regardless of actual quality.
- **Q:** How do you know a judge is trustworthy? → Measure its agreement against a human-labeled set before relying on it — a judge that disagrees with humans half the time looks rigorous but is worthless.
- **Q:** A field-extraction feature passed a dozen manual checks and two weeks in production, then extracted the wrong of two dates in one message — what was the actual root cause? → Nobody had written a graded case for a two-date input; every manual check used single-date messages, so validation (which passed) never tested for the *right* value, only a well-formed one.
- **Q:** Which test level catches most silent production breaks, and why? → Integration — the seam where two components hand off (e.g. retrieval → model call); each side can pass its own unit/functional test while the handoff between them is broken.
- **Q:** What does a trace add that a failing test alone doesn't? → *Where* the failure happened — a timeline of steps, tool calls, and timing, turning "something's wrong" into "step 4: parser raised KeyError."
- **Q:** The one question that classifies any failure as retrievable or terminal? → Would waiting and retrying the identical request plausibly work?
- **Q:** Default when you're unsure whether a failure is retrievable or terminal? → Terminal — a misclassified terminal failure fails loudly and gets fixed; a misclassified retrievable failure hammers the service and hides the real problem.
- **Q:** Should you write your own retry loop on top of the SDK's built-in retries? → Not both uncoordinated — decide explicitly: let the SDK own transient retries and use your code for app-specific fallbacks, or turn SDK retries down and own the full path yourself.
- **Q:** What's more precise than exponential backoff for a 429/529? → Honor the `retry-after` header when present — it's the service telling you exactly when capacity returns; fall back to backoff only when it's absent.
- **Q:** Is a refusal retrievable? → No — it's HTTP 200 with `stop_reason: "refusal"`, a content decision not a transient fault. The status-code retrievable classifier won't catch it; check `stop_reason` separately.
- **Q:** What happens if a tool call fails and your code returns an empty result instead of `is_error: true`? → The model treats the empty result as valid data and reasons on top of it — a confident but wrong answer, harder to catch than a visible failure.

Model Selection and Optimization

- **Q:** Why can't you rely on a fixed chars-per-token constant? → The ratio depends on the model's tokenizer, which changes across generations (e.g. Fable 5's tokenizer produces ~30% more tokens than pre-Opus-4.7 models for the same text).
- **Q:** Two distinct context-window failure modes? → Input already too large → rejected with a validation error before generation. Input fits but hits the ceiling mid-generation → stops with `model_context_window_exceeded`, partial output returned.
- **Q:** Do the newest Claude models accept `temperature` / `top_p` / `top_k`? → No — setting them returns a 400 error; behavior is steered through prompting instead.

- **Q:** Does `temperature: 0` guarantee identical output across calls? → No — more repeatable, not identical.
- **Q:** Why do exact-text test assertions fail on Claude output? → Non-determinism — the same correct answer can be expressed many ways; assert on the property (field present, value in range, parses), not exact text.
- **Q:** Zero-shot vs. one-shot vs. multi-shot? → Zero-shot: instruction only. One-shot: one example. Multi-shot/few-shot: several examples — shows exact shape a description alone can't pin down.
- **Q:** Sync vs. streaming vs. async client vs. Batch API — one line each? → Sync: wait for full response. Streaming: response in pieces over SSE as generated. Async client: non-blocking concurrency, same real-time latency. Batch: bulk, up to 24h, lower cost.
- **Q:** The practical model-selection workflow? → Start with Sonnet; move up a tier only when an eval shows it missing the quality bar; move down to Haiku only when an eval shows the drop is acceptable.
- **Q:** What replaced `budget_tokens` for tuning reasoning depth, and what happens if you still use `budget_tokens` on newest models? → The `effort` parameter (low/medium/high/xhigh/max); `budget_tokens` is deprecated and returns a 400 error on the newest generations.
- **Q:** Cache TTL choices and the tradeoff? → 5-minute (default) or 1-hour (survives longer gaps, higher cache-write price).
- **Q:** Cache breakpoint limits? → Up to 4 explicit breakpoints per request, 20-block lookback window, minimum token threshold per block.
- **Q:** Approximate cache write/read pricing multipliers vs. base input tokens? → Writes: ~1.25x (5-min TTL) or ~2x (1-hour TTL). Reads: ~0.1x. Economics only work when reads outnumber writes.
- **Q:** What's the "most common and most expensive model-selection mistake in production"? → Reaching for the most capable model by default without an eval forcing the question — ignoring that a wrong answer from a cheaper model can cost more downstream than the tier upgrade would have.
- **Q:** Image token cost formula? → $\lceil \text{width}/28 \rceil \times \lceil \text{height}/28 \rceil$ visual tokens (28×28-pixel patches). A 1000×1000px image ≈ 1,296 tokens.
- **Q:** Why measure image token cost against production images before shipping? → The fix (resize) is a 10-minute change at design time and considerably more expensive to retrofit after deployment — ten high-res screenshots can consume as much context as a detailed system prompt.
- **Q:** Batch API request/size limit per call? → Up to 100,000 requests or 256MB, whichever hits first.
- **Q:** Do Batch API results come back in submission order? → No — arbitrary order; use `custom_id` on each request to match results back to inputs.
- **Q:** Why doesn't chunking a loop over the synchronous endpoint fix rate-limit errors? → It's not batching — the API still sees one request per item, back to back; the Batch API is a different submission model, not a smaller batch size.

Prompt and Context Engineering

- **Q:** What's the fix for a prompt that "worked once" but breaks in production — more words? → No — diagnose which *structural* technique is missing and add exactly that, not more wording.
- **Q:** Symptom "wrong output shape" → missing technique? → An output constraint.
- **Q:** Symptom "content drifts / scope creeps across turns" → missing technique? → A system prompt, or a more specific one.
- **Q:** Symptom "right task, invented structure" → missing technique? → Few-shot examples.
- **Q:** Symptom "clean on tested inputs, breaks on a variant" → missing technique? → A constraint covering that specific variant.
- **Q:** Why do XML tags matter in a few-shot prompt? → They mark where each example starts/ends so Claude doesn't read the examples as part of the live instruction.

- **Q:** Prune/clear vs. compaction — when does each apply? → Prune: bloat is re-fetchable tool output (cheap, lossless). Compaction: bloat is dialogue/reasoning that can't be re-fetched (LLM summarization, more expensive).
- **Q:** Where should persistent rules live if compaction might drop early prompt instructions? → CLAUDE.md — it's re-injected on every request, unlike the initial prompt.
- **Q:** JSON outputs vs. strict tool use — what does each constrain? → JSON outputs: the final response. Strict tool use: the arguments Claude passes to a tool.
- **Q:** Why is a schema constraint stronger than a prompt-level "return only JSON" instruction? → The API enforces it on every generated token (constrained decoding) rather than trusting the model to remember an instruction — can't slip on an untested edge case.
- **Q:** Two cases where a "guaranteed" schema still doesn't parse? → Refusal (`stop_reason: "refusal"`) and truncation (`stop_reason: "max_tokens"`).
- **Q:** Cost of enabling structured outputs on a schema that changes often? → Repeated first-request grammar-compile latency — compiled grammars are cached only 24h from last use.
- **Q:** Can you combine JSON outputs with message prefilling? → No — mutually exclusive on the same request.

Security and Safety

- **Q:** Direct vs. indirect prompt injection — who's the adversary? → Direct: the application's own user. Indirect: a trusted user, but Claude processes third-party content (email, webpage, tool result) containing adversarial instructions.
- **Q:** The core structural fix for indirect injection? → Isolate untrusted content — put it only in `tool_result` blocks, never `system` /plain `user` text.
- **Q:** Why JSON-encode untrusted content? → Gives unambiguous delimiters so an attacker can't close a quote/tag and "break out" into an instruction context.
- **Q:** Why shouldn't you put your own instructions inside a tool result? → Claude is trained to distrust that channel — your legitimate instructions could get discounted along with any injected ones. Send them in the following `user` turn instead.
- **Q:** What's a "harmlessness screen"? → A lightweight-model (e.g. Haiku 4.5) pre-classification of input/tool-output, using structured outputs to force a parseable verdict, before the content reaches the main conversation.
- **Q:** Is a guardrail sufficient on its own? → No — treat it as one layer in defense-in-depth, never a replacement for input validation, structured prompts, least privilege, or human approval on destructive actions.
- **Q:** `PreToolUse` hook exit codes — which one blocks? → Exit code 2 denies the call. Exit code 1 only warns without blocking — a common source of "the hook didn't stop it" bugs.
- **Q:** Does a `PreToolUse` deny hook still work under `bypassPermissions` / `--dangerously-skip-permissions`? → Yes — it's a separate, earlier gate than the permission-mode decision.
- **Q:** How many times can you retrieve a newly-created API key from the Console? → Once, at creation — capture it to a secrets manager immediately.
- **Q:** What's the advantage of Workload Identity Federation over a static API key? → Exchanges a platform-issued identity (e.g. GitHub Actions OIDC token) for a short-lived Anthropic access token — no static secret to create, store, or rotate.
- **Q:** A component that's trusted in isolation (passes its own tests) — is the seam leaving it automatically trustworthy? → No — a component passing its own tests has no seam-level controls; every point data crosses between deployment environments needs its own explicit boundary control.

- **Q:** In a multi-component app, which component's scoping determines the whole application's blast radius? → The most-privileged one (typically the one reaching an external system, like an MCP server) — the app is only as contained as its most privileged seam.
- **Q:** Attorney-client privilege constraint — what does it typically rule out in code? → Calls from a consumer-grade surface the firm can't audit end-to-end; Anthropic doesn't log conversation content by default on direct API traffic, so the org must implement its own logging.
- **Q:** Does a HIPAA BAA cover Anthropic's Console or beta features? → No — BAA coverage excludes Console, Workbench, beta features, and consumer plans; verify the current feature-eligibility list.
- **Q:** Does the direct Anthropic API provide EU data residency? → No — EU-residency requirements route through Bedrock or Vertex with region pinned, not the first-party API directly.
- **Q:** Is "Claude Enterprise on AWS Marketplace" FedRAMP authorized? → No — the three authorized routes are Claude for Government (C4G), Bedrock GovCloud, and Vertex AI Assured Workloads.

Tools and MCPs

- **Q:** With default `tool_choice: {"type": "auto"}`, when does Claude call a tool vs. respond directly? → Calls when the request maps to a described tool capability and the answer isn't already in context; responds directly for stable knowledge, creative tasks, conversation.
- **Q:** Client-side vs. server-side/harness tool dispatch? → Client-side: your app executes the logic and returns `tool_result`. Server-side/harness: the harness (Claude Code, Agent SDK loop) executes it as part of the loop.
- **Q:** Parallel tool use — which tools run concurrently vs. sequentially? → Read-only tools (Read, Glob, Grep, read-only MCP tools) run concurrently; state-mutating tools (Edit, Write, Bash) run sequentially to avoid conflicts.
- **Q:** How does a custom tool opt into parallel execution? → Set `readOnlyHint` in its annotations (shared field name from the MCP SDK).
- **Q:** Three MCP server capability types? → Resources (file-like data), Tools (callable functions), Prompts (reusable templates).
- **Q:** stdio vs. HTTP/remote transport for an MCP server? → stdio: local subprocess, stdin/stdout, simplest for single-machine setups. HTTP: reached over the network, needed when shared across users/machines.
- **Q:** Three MCP server registration scopes in Claude Code? → Local (personal, this project), Project (shared via `.mcp.json`, checked in), User (personal, global) — mirrors settings.json scoping.
- **Q:** Why does MCP tool search defer schema loading by default? → Loading every connected server's full tool schema upfront on every request can consume significant context before the agent does any work.
- **Q:** MCP connects Claude to __; Skills teach Claude __. → ...data (live, dynamic); ...what to do with that data (procedural knowledge/judgment).
- **Q:** Exam guide Sample Question 3 pattern — reusable internal API access across several apps, maintained independently → answer? → Build an MCP server exposing the operations as tools, not hard-coding logic per app or relying on a built-in tool.
- **Q:** Four message block types in a tool-use conversation, and which one is uniquely irreversible once modified? → text, tool_use, tool_result, thinking. `thinking`: its signature verifies the reasoning hasn't been edited — any modification (even a summary) breaks the signature and the API rejects the message.
- **Q:** What breaks a request if a `tool_use` block's response arrives in a later turn instead of the immediately following one? → The API rejects it at validation — this is structural, not something a prompt change can fix.
- **Q:** The two-part pattern every tool description needs? → When to use it, AND when not to — "use this to find information" alone can't be distinguished from any other retrieval tool.

- **Q:** Two tools with near-identical descriptions cause Claude to route to the wrong one repeatedly — what's the fix? → Add one exclusion sentence to each description naming when *not* to call it. If they still can't be cleanly separated, merge into one tool with a `type` parameter.
- **Q:** When should you use `disable_parallel_tool_use`? → When one tool's output feeds the next call's input — a real dependency needs separate turns, since the second call can't be built until the first result is back.
- **Q:** `mcp_toolset`'s `defer_loading` vs. `enabled` — what does each control? → `defer_loading`: delays loading a tool's definition until actually needed (context-cost lever). `enabled`: turns individual tools on/off so a server can be registered but only a subset exposed.